Step 3 — A Retriever Agent That Judges Its Own Search Results — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 3: the first real searching agent. The Retriever searches your documents with Doc08's retriever — and then **judges** what it found against a written-down distance threshold, instead of passing along "the closest thing" no matter how far off it actually is. A question with no real match (every chunk's distance is past the threshold) goes to an honest "not found" path; questions that aren't about the documents skip the search entirely via keyword routing. The Reasoner's file and node also appear here, starting as simply as possible: it quotes the best excerpt verbatim, no model call yet. **If not:** every question would get an answer built on whatever the search happened to return, grounded or not.

After this step document questions are answered from your documents (still approved by you). `answer_node`, `approval_node` and `main.py` don't change. Files not shown here don't change.

The RAG files

Story — `rag/chunking.py`, `rag/ingest.py`, `rag/retriever.py`: this is Doc08's Build Task retriever, with two small changes: the OpenAI client is created *inside* each function (so the key from `.env` is loaded first, like the logger), and `build_vector_store()` logs what it ingested. **If not:** you'd rewrite chunking, embedding and search from scratch — and lose Doc08's safe delete-then-rebuild.

```
# src/rag/chunking.py
# Doc08's two chunkers: chunk_by_chars() and chunk_by_paragraph().
def chunk_by_chars(text, chunk_size=300):
    chunks: list[str] = []
    i = 0
    while i < len(text):
        chunks.append(text[i:i + chunk_size])
        i += chunk_size
    return chunks

def chunk_by_paragraph(text):
    raw_pieces = text.split("\n\n")
    chunks = []
    for piece in raw_pieces:
```

```
        stripped = piece.strip()
        if stripped != "":
            chunks.append(stripped)
    return chunks
```

```
# src/rag/ingest.py
# build_vector_store(): deletes and rebuilds the Chroma collection, chunks
# every .md/.txt file, embeds all chunks in one call, and stores them with
# their file name and position. (Doc08's Build Task file.)
from pathlib import Path

import chromadb
from chromadb.errors import NotFoundError
from openai import OpenAI

from src.rag.chunking import chunk_by_paragraph
from src.utils.logger import get_logger

EMBEDDING_MODEL = "text-embedding-3-small"

chroma_client = chromadb.Client()

def build_vector_store(doc_folder, chunk_fn=chunk_by_paragraph,
                       collection_name="docs"):
    logger = get_logger(__name__)
    client = OpenAI()
    try:
        chroma_client.delete_collection(name=collection_name)
    except NotFoundError:
        pass
    collection = chroma_client.create_collection(name=collection_name)

    all_texts: list[str] = []
    all_metadatas: list[dict] = []
    all_ids: list[str] = []

    for path in Path(doc_folder).glob("*"):
        if path.suffix not in (".txt", ".md"):
            continue
        text = path.read_text()
        chunks = chunk_fn(text)
        for chunk_index, chunk_text in enumerate(chunks):
            all_texts.append(chunk_text)
            all_metadatas.append({
                "source_file": path.name, "chunk_index": chunk_index,
            })
```

```
            all_ids.append(f"{path.name}-{chunk_index}")

    if not all_texts:
        logger.warning(f"no documents found in {doc_folder}")
        return collection

    response = client.embeddings.create(model=EMBEDDING_MODEL, input=all_texts)
    embeddings = []
    for item in response.data:
        embeddings.append(item.embedding)

    collection.add(
        ids=all_ids, embeddings=embeddings,
        documents=all_texts, metadatas=all_metadatas,
    )
    logger.info(f"ingested {len(all_texts)} chunks from {doc_folder}")
    return collection
```

```
# src/rag/retriever.py
# retrieve(collection, query, k): the k closest chunks as dicts with text,
# source_file, chunk_index and score. (Doc08's Build Task file.)
from openai import OpenAI

from src.rag.ingest import EMBEDDING_MODEL

def retrieve(collection, query, k=3):
    if collection.count() == 0:
        return []

    client = OpenAI()
    response = client.embeddings.create(model=EMBEDDING_MODEL, input=[query])
    query_embedding = response.data[0].embedding

    results = collection.query(query_embeddings=[query_embedding], n_results=k)

    texts = results["documents"][0]
    metadatas = results["metadatas"][0]
    distances = results["distances"][0]

    documents: list[dict] = []
    for text, metadata, distance in zip(texts, metadatas, distances):
        documents.append({
            "text": text,
            "source_file": metadata["source_file"],
            "chunk_index": metadata["chunk_index"],
            "score": distance,
```

```
        })

    return documents
```

Story — `rag/store.py`: this builds the vector store **once**, the first time a node needs it, and keeps the reference for every later call — a small lazy-singleton class, Doc07's "state holder" pattern. **If not:** building at import time would run before the key is loaded, and building fresh inside the node would re-embed every document for every single question.

```
# src/rag/store.py
# get_collection(): builds the vector store the first time a node needs it,
# then reuses it — not at import time, not once per question.
from src.rag.ingest import build_vector_store

class StoreHolder:
    def __init__(self):
        self.collection = None

store = StoreHolder()

def get_collection():
    if store.collection is None:
        store.collection = build_vector_store("data/docs")
    return store.collection
```

Also create an empty `rag/__init__.py`, and write your documents into `data/docs/` — 5 short `.md` files, each a few paragraphs, **no heading lines** (a heading alone becomes its own useless chunk). For example:

`data/docs/refund_policy.md`

```
Customers can return any product within 30 days of purchase for a full
refund. The original receipt or order number is required.

Refunds are paid back to the original payment method within 5 working
days after the returned product arrives at our warehouse.
```

The other four documents in the example set:

`data/docs/shipping.md`

```
Orders are shipped within 2 working days. Standard delivery takes
3 to 5 working days, and express delivery takes 1 to 2 working days.
```

```
Shipping is free on orders over 50 dollars. Every order gets a
tracking number by email as soon as it leaves our warehouse.
```

`data/docs/warranty.md`

```
Every product has a 1-year warranty against manufacturing faults.
The warranty starts on the day the product is delivered.

The warranty does not cover damage from drops, water or misuse. To
make a claim, send the order number and a photo of the fault.
```

`data/docs/account_security.md`

```
Passwords must be at least 12 characters long. Support staff will
never ask for your password, by email, chat or phone.

After 5 failed login attempts, the account is locked for 15 minutes.
Two-factor authentication can be turned on in the account settings.
```

`data/docs/pricing.md`

```
The Basic plan costs 9 dollars a month and the Pro plan costs 29
dollars a month. Both plans can be cancelled at any time.

Paying yearly gives two months free. Prices include tax for customers
in the European Union.
```

Story — `agents/reasoner_agent.py` : this is the Reasoner's file and node, starting as simply as possible: `reason_node` answers by quoting the best-matching excerpt word for word, plus its source file name — no model call involved yet. Step 4 keeps this exact node name and upgrades what runs inside it to a real model call. **If not:** a model-written answer would arrive at the same time as the search, and a bad answer couldn't be pinned on either one.

```
# src/agents/reasoner_agent.py
# The Reasoner agent: reason_node() answers only from the found excerpts.
# For now it quotes the best excerpt and names its file.

def reason_node(state):
    best = state["sources"][0]
    answer = best["text"] + " [" + best["source_file"] + "]"
    return {"answer": answer, "path": ["reason_node"]}
```

> Back to this step · Hint 1 · Hint 2 · Solution

Solution — a written relevance bar, and an honest "not found" path

Changed from Step 2: `state.py` adds `sources` and `grounded`.

Story — `state.py`: the found chunks live **in the state** itself, not in a local variable that disappears between nodes. Because the state is what the checkpointer saves, a run paused for approval keeps its evidence intact, and resuming never searches again — the chunks are already there. **If not:** a resumed run would have its answer but no record of where it came from.

```
# src/state.py
# DocState: everything any node can read or write. `path` adds each node's
# entries to the list (operator.add), so it logs the nodes that ran.
# start_state(task) gives every field a starting value.
import operator
from typing import Annotated, TypedDict

class DocState(TypedDict):
    task: str
    path: Annotated[list[str], operator.add]
    sources: list[dict]
    grounded: bool
    approved: bool
    answer: str

def start_state(task):
    return {
        "task": task,
        "path": [],
        "sources": [],
        "grounded": False,
        "approved": False,
        "answer": "",
    }
```

Story — `agents/retriever_agent.py`: four small functions do the whole job. `route_search()` decides search-or-skip by keyword, before any node runs (Doc10's pattern). `is_relevant()` is the relevance bar itself: the **best** chunk's distance must be at most `MAX_DISTANCE` (lower distance means closer match, Doc08's convention). `retrieve_node()` runs the search and records the verdict in state. `no_grounding_node()` returns the honest "not found" answer when nothing clears

the bar. **If not:** "grounded" would mean "the search returned something" — which is always true, since a vector search always returns its closest chunks.

```
# src/agents/retriever_agent.py
# The Retriever agent: route_search() decides search or skip,
# retrieve_node() searches, is_relevant() judges the best score against
# MAX_DISTANCE (a distance: lower is closer), and no_grounding_node() gives
# an honest "not found".
from src.rag.retriever import retrieve
from src.rag.store import get_collection
from src.utils.logger import get_logger

SEARCH_KEYWORDS = [
    "document", "policy", "refund", "return", "shipping", "delivery",
    "warranty", "password", "account", "price", "plan",
]

MAX_DISTANCE = 1.2

def route_search(state):
    task = state["task"].lower()
    for keyword in SEARCH_KEYWORDS:
        if keyword in task:
            return "search"
    return "skip"

def is_relevant(sources):
    if not sources:
        return False
    best = sources[0]["score"]
    for source in sources:
        if source["score"] < best:
            best = source["score"]
    return best <= MAX_DISTANCE

def retrieve_node(state):
    logger = get_logger(__name__)
    sources = retrieve(get_collection(), state["task"], k=3)
    grounded = is_relevant(sources)
    for source in sources:
        logger.info(
            f"found {source['source_file']} (score {source['score']:.2f})"
        )
    return {"sources": sources, "grounded": grounded,
            "path": ["retrieve_node"]}

def has_grounding(state):
```

```
    if state["grounded"]:
        return "reason"
    return "no_grounding"

def no_grounding_node(state):
    answer = "I couldn't find this in your documents, so I won't guess."
    return {"answer": answer, "path": ["no_grounding_node"]}
```

Changed from Step 2: two conditional edges and three new nodes. General answers and "not found" end the run; only a document answer goes to `approval_node`. This is the graph's **final** shape — Steps 4 and 5 don't change this file.

Story — `graph.py`: `add_conditional_edges(node, function, map)` is the decision point — the routing function's return value picks the next node from the mapping you supply (Doc09's pattern). Here it routes a grounded answer toward approval and an honest "not found" answer straight to the end. **If not:** every path would look the same, and a "not found" answer would wait for human approval for no real reason.

```
# src/graph.py
# build_graph(): adds the nodes and edges and compiles the graph. The
# checkpointer is passed in, so the app and the tests can each choose one.
from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import END, START, StateGraph

from src.agents.approval_agent import approval_node
from src.agents.reasoner_agent import reason_node
from src.agents.retriever_agent import (
    has_grounding,
    no_grounding_node,
    retrieve_node,
    route_search,
)
from src.nodes import answer_node, clean_task
from src.state import DocState

def build_graph(checkpointer=None):
    if checkpointer is None:
        checkpointer = MemorySaver()

    builder = StateGraph(DocState)
    builder.add_node("clean_task", clean_task)
    builder.add_node("answer_node", answer_node)
    builder.add_node("retrieve_node", retrieve_node)
    builder.add_node("no_grounding_node", no_grounding_node)
    builder.add_node("reason_node", reason_node)
    builder.add_node("approval_node", approval_node)
```

```
    builder.add_edge(START, "clean_task")
    builder.add_conditional_edges(
        "clean_task", route_search,
        {"search": "retrieve_node", "skip": "answer_node"},
    )
    builder.add_conditional_edges(
        "retrieve_node", has_grounding,
        {"reason": "reason_node", "no_grounding": "no_grounding_node"},
    )
    builder.add_edge("reason_node", "approval_node")
    builder.add_edge("approval_node", END)
    builder.add_edge("answer_node", END)
    builder.add_edge("no_grounding_node", END)
    return builder.compile(checkpointer=checkpointer)
```

Story — `tests/unit/test_chunking.py` + `tests/unit/test_routing.py`: these check the chunkers and the routing/relevance rules with hand-made inputs and no API key needed — pure logic, instant to run. One function per rule, each printing `OK` or `WRONG`. **If not:** a wrong `MAX_DISTANCE` comparison (`>` instead of `<=`) would only show up later as strange, ungrounded answers.

```
# tests/unit/test_chunking.py
# Checks Doc08's two chunkers. Each case prints OK or WRONG.
# No API calls, no API key needed.
# Run from the repo root: python -m tests.unit.test_chunking
from src.rag.chunking import chunk_by_chars, chunk_by_paragraph

def test_chunk_by_chars():
    lengths = []
    for chunk in chunk_by_chars("a" * 650, chunk_size=300):
        lengths.append(len(chunk))
    result = "OK"
    if lengths != [300, 300, 50]:
        result = "WRONG"
    print(f"case 1 (650 chars, size 300) -> {result}: {lengths}")

def test_chunk_by_paragraph():
    text = "First paragraph.\n\n\n\nSecond paragraph.\n\n   \n\n"
    chunks = chunk_by_paragraph(text)
    result = "OK"
    if chunks != ["First paragraph.", "Second paragraph."]:
        result = "WRONG"
    print(f"case 2 (empty pieces dropped) -> {result}: {chunks}")

if __name__ == "__main__":
```

```
    test_chunk_by_chars()
    test_chunk_by_paragraph()
```

```
# tests/unit/test_routing.py
# Checks the Retriever agent's rules with hand-made states and scores.
# Each case prints OK or WRONG. No API calls, no API key needed.
# Run from the repo root: python -m tests.unit.test_routing
from src.agents.retriever_agent import (
    has_grounding,
    is_relevant,
    route_search,
)

def show(case, passed):
    result = "OK"
    if not passed:
        result = "WRONG"
    print(f"{case} -> {result}")

def test_document_question_searches():
    task = {"task": "What is the refund policy?"}
    show("case 1 (refund question searches) ", route_search(task) == "search")

def test_general_question_skips():
    task = {"task": "What is the capital of France?"}
    show("case 2 (general question skips)   ", route_search(task) == "skip")

def test_close_match_is_relevant():
    sources = [{"score": 1.6}, {"score": 0.4}]
    show("case 3 (best score 0.4 counts)    ", is_relevant(sources))

def test_far_match_is_not_relevant():
    sources = [{"score": 1.7}, {"score": 1.9}]
    show("case 4 (best score 1.7 doesn't)   ", not is_relevant(sources))

def test_nothing_found():
    show("case 5 (no sources at all)        ", not is_relevant([]))

def test_grounded_goes_to_reasoner():
    choice = has_grounding({"grounded": True})
    show("case 6 (grounded -> reason)       ", choice == "reason")

def test_not_grounded_says_so():
    choice = has_grounding({"grounded": False})
    show("case 7 (not grounded -> honest no)", choice == "no_grounding")
```

```
if __name__ == "__main__":
    test_document_question_searches()
    test_general_question_skips()
    test_close_match_is_relevant()
    test_far_match_is_not_relevant()
    test_nothing_found()
    test_grounded_goes_to_reasoner()
    test_not_grounded_says_so()
```

Changed from Step 2: cases 5 and 6 are added; cases 1-4 stay the same.

Story — `tests/integration/test_live_graph.py`: this checks the two new paths against the real API: a document question that gets searched and comes back grounded (case 5), and a question the documents don't cover, which must route all the way to `no_grounding_node` (case 6). **If not:** "the not-found path works" would be a hope, not something verified against the real search and model.

```
# tests/integration/test_live_graph.py
# Runs the whole graph against the real OpenAI API — each step adds cases.
# Each case prints OK or WRONG. Needs a valid OPENAI_API_KEY in .env.
# Run from the repo root: python -m tests.integration.test_live_graph
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import Command

from src.config import load_config
from src.graph import build_graph
from src.state import start_state

ACTION_TASK = "What is the refund policy? Send it to the customer."

def run(graph, thread_id, task, decision="yes"):
    config = {"configurable": {"thread_id": thread_id}}
    outcome = graph.invoke(start_state(task), config)
    if decision is not None and "__interrupt__" in outcome:
        outcome = graph.invoke(Command(resume=decision), config)
    return outcome

def show(case, passed, outcome):
    result = "OK"
    if not passed:
        result = "WRONG"
    print(f"{case} -> {result}: {outcome.get('path')}")

def test_general_question(graph):
    outcome = run(graph, "t1", "What is the capital of France?")
    passed = "Paris" in outcome["answer"] and "answer_node" in outcome["path"]
```

```
    show("case 1 (general question)", passed, outcome)

def test_action_pauses(graph):
    outcome = run(graph, "t2", ACTION_TASK, decision=None)
    show("case 2 (pauses)          ", "__interrupt__" in outcome, outcome)

def test_human_says_yes(graph):
    outcome = run(graph, "t3", ACTION_TASK, "yes")
    passed = outcome["approved"] and "human_approved" in outcome["path"]
    show("case 3 (human: yes)      ", passed, outcome)

def test_human_says_no(graph):
    outcome = run(graph, "t4", ACTION_TASK, "no")
    passed = outcome["answer"].startswith("Not approved")
    show("case 4 (human: no)       ", passed, outcome)

# new in Step 3
def test_grounded_answer(graph):
    outcome = run(graph, "t5", "What is the refund policy?")
    passed = outcome["grounded"] and "retrieve_node" in outcome["path"]
    show("case 5 (grounded)        ", passed, outcome)

# new in Step 3
def test_not_in_documents(graph):
    outcome = run(graph, "t6", "What is the policy on remote work?")
    passed = outcome["path"][-1] == "no_grounding_node"
    show("case 6 (not found)       ", passed, outcome)

if __name__ == "__main__":
    load_config()
    graph = build_graph(MemorySaver())
    test_general_question(graph)
    test_action_pauses(graph)
    test_human_says_yes(graph)
    test_human_says_no(graph)
    # new in Step 3
    test_grounded_answer(graph)
    test_not_in_documents(graph)
```

Run everything:

```
python -m tests.unit.test_config
python -m tests.unit.test_nodes
python -m tests.unit.test_chunking
python -m tests.unit.test_routing
```

```
python -m tests.integration.test_live_graph
python main.py
```

Expected output of the two new unit tests:

```
case 1 (650 chars, size 300) -> OK: [300, 300, 50]
case 2 (empty pieces dropped) -> OK: ['First paragraph.', 'Second
paragraph.']
```

```
case 1 (refund question searches)  -> OK
case 2 (general question skips)    -> OK
case 3 (best score 0.4 counts)     -> OK
case 4 (best score 1.7 doesn't)    -> OK
case 5 (no sources at all)         -> OK
case 6 (grounded -> reason)        -> OK
case 7 (not grounded -> honest no) -> OK
```

Expected output of the integration test — cases 1-4 as in Step 2 (case 2-4's path now goes through `retrieve_node` and `reason_node`, because the action task is about the refund policy), then:

```
case 5 (grounded)         -> OK: ['clean_task', 'retrieve_node',
'reason_node', 'human_approved']
case 6 (not found)        -> OK: ['clean_task', 'retrieve_node',
'no_grounding_node']
```

(Long lines are wrapped here to fit the page.) To tune `MAX_DISTANCE` for your own documents, watch the `found ... (score ...)` log lines for a few good and bad questions, and set the bar between them.